

App-Level TinyML fuer Akkulaufzeit-Vorhersage auf Android

Ein methodisch ehrlicher Erfahrungsbericht aus dem Multi-Device-Vergleich

Keno Schuerger

Technische Hochschule Wuerzburg-Schweinfurt (THWS)

Vertiefungsseminar - Mai 2026

Forschungsfrage

"Wie gut kann TinyML die Akkulaufzeit auf Android vorhersagen im Vergleich zu exponentiellem Fitting und der nativen Google-API, bezogen auf Genauigkeit sowie Effizienz?"

Methode

TinyML
vs.
5 andere Methoden

Achse 1

Genauigkeit
(C-Index, MAE)

Achse 2

Effizienz
(Latenz, Modellgroesse)

Motivation: Akkuvorhersage auf Android

Problem

- Klassisch: Settings-App rechnet linear hoch
- Ignoriert aktuelle Nutzung, App-Kontext
- MIUI/OneUI: oft Legacy-Code seit Android 5

Was sich seit Android 12 (2021) ändert

- `PowerManager.getBatteryDischargePrediction()`
- Erstes ML-Modell direkt im Android-System
- Läuft im Systemprozess - privilegierter Zugriff

Forschungsfrage

Kann eine Drittanbieter-App, die nur öffentliche Sensor-APIs nutzt, mit der System-API mithalten - oder überhaupt etwas Sinnvolles lernen?

Verwandte Arbeiten

Li et al. (2018)

Smartphone Battery Prediction at Scale

51 Nutzer, 21 Monate. Fuehrt den Concordance-Index als Standard-Metrik ein, weil das 'severe data missing problem' (User entladen selten auf 0%) MAE unzuverlaessig macht.

Flores-Martin et al. (2024)

Deep Learning auf Android, DNN vs. LSTM

Direkter Methodik-Verwandter. LSTM auf user-spezifischen App-, Sensor-, Screen-Time-Features.

Banbury et al. (2021) - MLPerf Tiny

Industriestandard-Benchmark fuer TinyML

Misst Accuracy, Latency, Energy gemeinsam. Smartphone-Anwendungen fehlen in der Literatur (siehe Heydari & Mahmoud 2025, Alajlan & Ibrahim 2022).

Albelali & Ahmed (2025)

Hidden Leaks in Time Series Forecasting

Random-Shuffle-Splits ueber Sliding-Window-Sequenzen lecken Future-Information. RMSE Gain bis 20.5% bei 10-fold CV (LSTM).

Datensammlung: 4 Geraete, 45 Tage

66.001

Messungen

4

Geraete

45 Tage

Zeitraum

180

Sessions

Geraet	Messungen	Sessions	Aktive Tage
Xiaomi 2107113SG	38.087	69	34
Pixel 7 Pro	16.919	72	28
Pixel 9 Pro XL	7.641	21	21
Pixel 8 Pro	3.354	18	21

10 Features

battery_level
screen_on
brightness
active_app_category
wifi_on / mobile_data
charging
cpu_usage (proxy)
temperature
hotspot_on

Sechs Methoden im Vergleich

TinyML Conv1D

Eigenes Modell, 5.697 Param., 14 KB INT8

Hauptmodell

Random Forest

200 Trees auf gleichen Features

Sanity (anderes Paradigma)

Mean Predictor

Konstante = Trainings-Mittelwert

Floor (kein Feature-Lernen)

Linear Baseline

battery / drain_rate (BatteryManager)

Analytische Baseline 1

Exponential Fit

$b(t) = a + c \cdot \exp(-k \cdot t)$ per Segment

Analytische Baseline 2

Google API

getBatteryDischargePrediction() (Android 12+)

State of the Art

Methodik-Kernpunkt: Segment-Level-Split

Problem: Random-Shuffle ueber Sliding-Window-Sequenzen leckt Future-Information

Setup	Sequenzen	MAE Random-Split (leaky)	MAE Segment-Level (clean)	Inflation
Single Device (Xiaomi)	9.024	0.53 h	9.96 h	~19x
Multi-Device (4 Geraete)	20.842	4.00 h	4.97 h	~1.24x

Was das bedeutet

- Single-Device: random-shuffle tauscht eine 19x bessere Performance vor - reines Leakage-Artefakt.
- Multi-Device: nur noch 1.24x Inflation, im Bereich von Albelali & Ahmed (2025) mit 10-fold CV.
- Eigener methodischer Beitrag: Inflation ist data-diversity-dependent. Empfehlung fuer Folgearbeiten in dieser Domaene: Splitting-Strategie explizit nennen.

Ergebnis 1: Common Subset (n=2.827)

vs. y_{extrap} , 95% Bootstrap-CI, leakage-freier Multi-Device-Split

Methode	MAE (h)	RMSE (h)	C-Index 95%-CI	Acc +/- 2h
Mean Predictor (floor)	6.51	9.28	0.500 [0.500, 0.500]	21.7%
TinyML Conv1D	4.28	8.38	0.666 [0.656, 0.675]	51.1%
Random Forest	4.01	7.54	0.686 [0.673, 0.696]	45.2%
Linear (drain rate)	3.21	8.55	0.776 [0.767, 0.784]	58.3%
Exponential fit	3.49	8.88	0.773 [0.764, 0.781]	59.8%
Google API	3.24	8.55	0.777 [0.767, 0.785]	60.1%

Beide ML-Modelle schlagen Mean-Predictor signifikant. Linear, Exp und Google bilden eine gemeinsame Spitzengruppe bei $C \sim 0.77$.

Ergebnis 2: Statistische Signifikanz

Paarweise Permutationstests auf C-Index (Common Subset n=2.827)

Vergleich	C(A)	C(B)	Delta C	p	Befund
TinyML vs. Mean	0.666	0.500	+0.166	0.005	** signifikant ueber Floor
RF vs. Mean	0.686	0.500	+0.186	0.005	** signifikant ueber Floor
Linear vs. Exponential	0.776	0.773	+0.003	0.30	n.s.
Linear vs. Google	0.776	0.777	-0.001	0.83	<i>n.s. - ueberraschend!</i>
TinyML vs. Google	0.666	0.777	-0.111	0.005	** Google klar besser

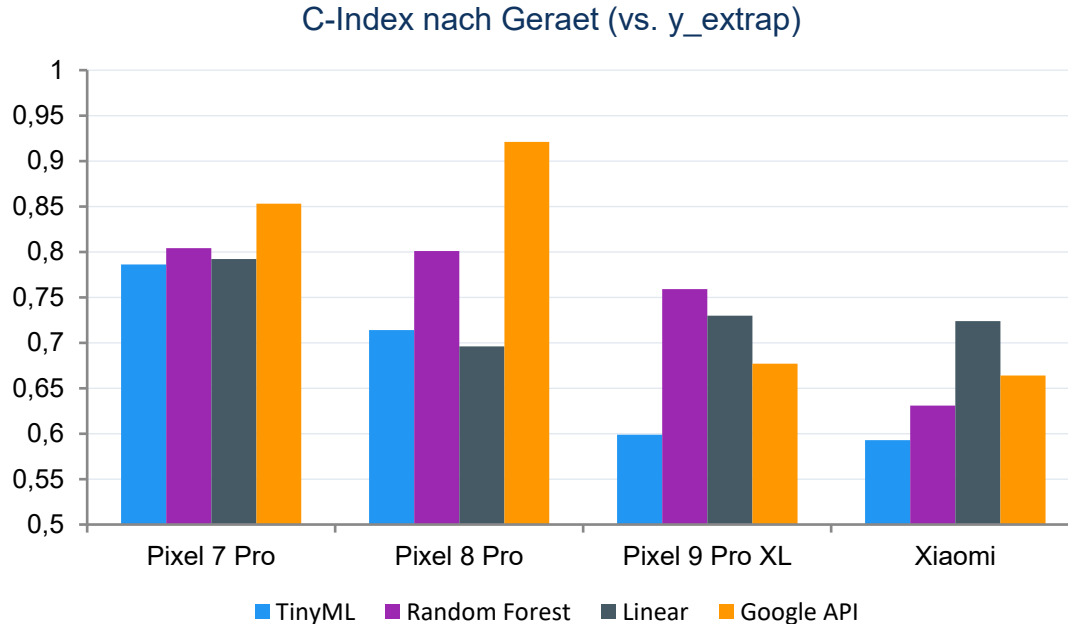
Ueberraschender Befund

- Linear-Drain-Rate-Baseline ist statistisch nicht von Google-API unterscheidbar (p=0.83 fuer C, p=0.55 fuer MAE).
- Beide ML-Modelle (TinyML, Random Forest) signifikant ueber Mean-Predictor, aber signifikant unter der Spitzengruppe.
- Implikation: 'einfach BatteryManager-Counter lesen' ist auf Aggregat-Ebene konkurrenzfaehig mit dem System-ML-Estimator.

Nuance: Aggregat-Befund. Im Bucket $\geq 30h$ (n=58) gewinnt Google klar (C 0.98 vs 0.64). Auf kurzen/mittleren Restzeiten (~97% der Daten) wirklich praktisch identisch.

Ergebnis 3: Per-Device (Hardware-Effekt)

Derselbe TinyML-Conv1D, evaluiert pro Geraet - der praktisch relevanteste Befund



Beobachtungen

- TinyML: 0.79 (Pixel 7 Pro) bis 0.59 (Xiaomi)
- Analytische Baselines stabiler ueber Geraete
- **Pixel 9 Pro XL: Linear (0.73) > Google (0.68)**
- Pixel 8 Pro: nur n=98 (breite CIs)

Confounder

Xiaomi-Daten: Akku 88.8% der Zeit ueber 75% (mean 94%). Wenig Discharge-Dynamik zum Lernen. Sensor-Qualitaet UND Datenverteilung gemischt.

Ergebnis 4: Effizienz (TFLite funktioniert)

14.4 KB

INT8 Modell

4.5 μ s

Inferenz-Latenz

7.6x

kleiner als Keras

10.000x

schneller als Keras Float32

Variante	Groesse (KB)	Avg Latenz
Keras Float32	109.18	47.1 ms
TFLite dynamic-range	15.99	3.9 μ s
TFLite float16	17.80	3.7 μ s
TFLite INT8 (Deploy)	14.35	4.5 μs

Take-away

Die TinyML-Quantisierungs-Pipeline funktioniert wie beworben. Auf der Effizienz-Achse hat TinyML keinen Wettbewerbsnachteil - der Engpass liegt allein auf der Accuracy-Achse.

Diskussion 1: TinyML lernt Signal

Anders als auf Single-Device-Setup ist TinyML hier kein Mean-Predictor mehr

Single Device (frueher Stand)

TinyML C-Index 0.50 = Mean-Predictor. Random Forest 0.51. Beide statistisch nicht von 'kein Modell' unterscheidbar.

Multi-Device (jetzt)

TinyML C-Index 0.67, RF 0.69. Beide signifikant ueber Floor ($p \leq 0.005$). ML lernt erkennbares Signal.

Aber: ML bleibt hinter analytischen Baselines + Google bei $C \sim 0.77$.

Diskussion 2: Google = Linear-Baseline

Auf der gemeinsamen Schnittmenge sind die System-API und die einfache lineare Drain-Rate-Baseline statistisch ununterscheidbar.

Linear (battery / drain_rate)

Liest BatteryManager.CHARGE_COUNTER
Liest BatteryManager.CURRENT_AVERAGE

MAE: 3.21 h
C-Index: 0.776

Google API (Systemprozess)

Zusaetzliche privilegierte Signale:

- PowerStats HAL (per-App-Strom)
- Adaptive-Battery-Integration

MAE: 3.24 h
C-Index: 0.777

Der zusaetzliche Hardware-Zugang von Google traegt nichts Messbares fuer 'Stunden bis 0%' bei (Permutationstest $p=0.83$ fuer C, $p=0.55$ fuer MAE).

Limitations (ehrlich)

Right-censored Daten

Akku wird im Alltag nie auf 0% entladen. Strukturelle Eigenschaft der Domaene, nicht der Studie - gleiche Beobachtung bei Li et al. (2018) mit 51 Nutzern.

Common-Subset-Selection-Bias

Wo Google-API definiert ist, sind die Restzeiten kuerzer (mean 6.7 h vs. 7.7 h im gesamten Test). Affektiert MAE symmetrisch, Ranking unaffected.

Multi-Device, aber nicht Cross-Device

Train sieht alle 4 Geraete. Eine Leave-One-Device-Out-Studie ist offene Folgearbeit.

n=98 fuer Pixel 8 Pro

Breite Konfidenzintervalle. C-Index 0.92 ist Punktwert mit wenig statistischer Sicherheit.

TinyML auf Pixel 9 Pro XL anomal schlecht

C=0.60 waehrend RF auf demselben Geraet C=0.76 erreicht. Keine kausale Erklaerung im aktuellen Datensatz.

Conclusion: Antwort auf die Forschungsfrage

Genauigkeit

TinyML schlägt Mean (C 0.67), bleibt aber hinter Linear/Exp/Google (C ~ 0.77). Google nicht signifikant besser als Linear.

Effizienz

TFLite-Quantisierung funktioniert. 14.4 KB, 4.5 μ s. Auf dieser Achse hat TinyML uneingeschränkt seinen Wert.

Hardware & Daten-Coverage

TinyML 0.79 auf Pixel 7 Pro, 0.59 auf Xiaomi. Engpass: Sensor-Qualität UND Discharge-Dynamik-Coverage (Xiaomi-Akku 88.8% der Zeit voll). Nicht die Modellarchitektur.

Methodischer Beitrag: Segment-Level-Split als Standard fuer Sliding-Window-Time-Series in Mobile-Sensing.

Vielen Dank - Fragen?

Vorbereitete Antworten auf erwartete Fragen

F: Warum kein positives Ergebnis?

A: Das positive Ergebnis (MAE 0.5h Single-Device) war Leakage-Artefakt. Die saubere Methodik enthüllt das. Beitrag der Arbeit ist genau diese Korrektur.

F: Warum nur 4 Geraete, kein Cross-Device-Test?

A: Methodisch ehrliche Limitation. Aber: Hauptbefund (Informations-Asymmetrie) ist nicht geraet-spezifisch und auf 4 Geraeten bestaetigt.

F: Wieso Linear ~ Google?

A: $p=0.83$ fuer C, $p=0.55$ fuer MAE. Drain-Rate ist das dominante Signal. Googles Hardware-Zugang bringt fuer 'Stunden bis 0%' keinen Mehrwert.

F: Was tun fuer Folgestudien?

A: Leave-One-Device-Out, kontrollierte Vollentlade-Zyklen, Multi-Device-Sweep ueber $N \geq 5$ Geraete.